Документация модуля: Student Dashboard (Панель студента - Главная страница)

Версия: 1.0
Описание: Модуль реализует главную страницу студента с функционалом SPA (Single Page Application). Обеспечивает навигацию по вкладкам без перезагрузки страницы, динамическое обновление URL и рендеринг контента (расписание, домашние задания) на основе JavaScript-данных.

Содержание

1. Архитектура и структура HTML
2. Система навигации (Routing)
3. Управление состоянием URL
4. Структура данных (Data Models)
5. Логика рендеринга (Rendering)
6. Руководство по расширению

---

1. Архитектура и структура HTML

Интерфейс построен на принципе вкладок (tabs). Весь контент загружается сразу (или через `include`), но отображается только активный блок.

Базовая разметка

```
<!-- Основной контейнер контента -->
<div class="main-content">

    <!-- Вкладка 1: Главная (Dashboard) -->
    <!-- ID должен соответствовать формату: tab-{tabName} -->
    <div id="tab-home" class="tab-content">
        <!-- Подключение шаблона Django -->
        {% include 'parties/student_dashboard/student_home.html' %}
    </div>

    <!-- TAB 2: ЛЕКЦИИ -->
    <div id="tab-lectures" class="tab-content hidden">
        ...
    </div>

</div>
```

Компоненты Dashboard (`student_home.html`)

Внутри главной вкладки используется сеточная верстка (`grid/flex`):

- **Header:** Приветствие и дата.
- **Stats Grid (`.stats-grid`):** 4 карточки со статистикой (Белсенді курстар, Орындалған тапсырмалар и т.д.).
- **Content Grid (`.dashboard-grid`):** Разделен на две колонки.
  - `.column-schedule`: Место для вставки расписания.
  - `.column-homework`: Место для вставки карточек ДЗ.

> ⚠ **Важно:** Содержимое колонок расписания и домашних заданий генерируется динамически через JS. В HTML файле эти блоки должны содержать только заголовки.

---

2. Система навигации (Routing)

Переключение между разделами осуществляется функцией `openTab(tabName)` (student_dashboard_base.js).

Файлы:

1.sidebar_student.html 2.student_dashboard.css 3.student_dashboard_base.js

Принцип работы `openTab`

Функция принимает строковый идентификатор вкладки и выполняет следующие действия:

> ⚠ **Важно:** Все элементы кроме элементов `.tab-content` находиться в sidebar_student.html. Элементы `.tab-content` находиться внутри `main.content-body`

1. **Скрытие вкладок:** Находит все элементы `.tab-content` и добавляет класс `.hidden`.
2. **Отображение активной:** Удаляет класс `.hidden` у элемента с `id="tab-{tabName}"`.
3. **Сайдбар:**
   - Находит все кнопки `.nav-btn` внутри `.sidebar`.
   - Удаляет класс `.active` у всех кнопок.
   - Добавляет класс `.active` кнопке, в чьем атрибуте `onclick` содержится текущий `tabName`.
4. **Заголовок:** Обновляет текст в `#page-header-title` согласно словарю `titles`.
5. **URL:** Вызывает `updateURLParameter` для смены GET-параметра `?page=...`.

Пример использования в HTML

Кнопка в боковой панели (sidebar):

```
<button class="nav-btn" onclick="openTab('home')">
    <i class="fa-solid fa-house"></i> Басты бет
</button>

<button class="nav-btn" onclick="openTab('homework')">
    <i class="fa-solid fa-book"></i> Үй тапсырмалары
</button>
```

---

3. Управление состоянием URL (student_dashboard_base.js)

Для сохранения состояния при перезагрузке страницы (F5) или отправке ссылки используется работа с `History API`.

Функция `updateURLParameter`

Обновляет GET-параметры без перезагрузки страницы.

```
/**
 * @param {string} key - Ключ параметра (например, 'page')
 * @param {string} value - Значение (например, 'schedule')
 */
function updateURLParameter(key, value) {
  const searchParams = new URLSearchParams(window.location.search);
  searchParams.set(key, value);
  const newPath = window.location.pathname + '?' + searchParams.toString();
  history.pushState(null, '', newPath);
}
```

Инициализация при загрузке

Скрипт проверяет URL при событии `DOMContentLoaded` для восстановления сессии пользователя.

```
document.addEventListener('DOMContentLoaded', function() {
    const urlParams = new URLSearchParams(window.location.search);
    const pageToOpen = urlParams.get('page'); // Получаем ?page=...

    if (pageToOpen) {
        openTab(pageToOpen); // Открываем конкретную вкладку
    } else {
        openTab('home');     // По умолчанию открываем 'home'
    }
});
```

---

4. Структура данных (Data Models) (home.js)

Данные хранятся в массивах объектов (JSON-формат). В будущем эти данные должны приходить через AJAX/Fetch запрос к API.

Модель: Расписание (`scheduleDataHome`)

```
[
    {
        time: "09:00",          // String: Время начала
        subject: "Киберқауіпсіздік", // String: Название предмета
```

```
        teacher: "Ахметов Е.К.", // String: ФИО преподавателя
        room: "А-305"           // String: Номер аудитории
    }
]
```

Модель: Домашнее задание (`homeworkDataHome`)

```
[
    {
        subject: "Алгоритмдер", // String: Предмет
        status: "Басталмаған",  // String: Текст статуса для пользователя
        statusColor: "gray",    // String: CSS-класс цвета (gray, yellow, green,
red)
        title: "Екілік іздеу...", // String: Описание задания
        dueDate: "28.11.2025"   // String: Дата дедлайна
    }
]
```

---

5. Логика рендеринга (Rendering)

Скрипт автоматически заполняет DOM-дерево на основе массивов данных.

Генерация расписания

1. Целевой контейнер: `.column-schedule` внутри `#tab-home`.
2. Для каждого элемента создается `div.list-card.schedule-item`.
3. Используется `innerHTML` для вставки иконок FontAwesome.

Генерация домашних заданий

1. Целевой контейнер: `.column-homework` внутри `#tab-home`.
2. Для каждого элемента создается `div.list-card`.